Assignment 6

Implementation of Data Integrity Constraints

Primary keys, foreign keys, UNIQUE, CHECK, DEFAULT, and verification

Project: LibSphere - Library Management System **Prepared:** 23 October 2026

Platform assumption: MySQL 8.0.16 or later (InnoDB). Outputs in this report are expected results derived from the included sample dataset; run the SQL in MySQL Workbench to capture authentic execution screenshots if required by the instructor.

Constraints put important rules in the database instead of relying only on the application interface. The DDL in Assignment 5 applies these constraints to all six tables; this report describes their purpose and gives test statements for successful and rejected cases.

1. Constraint inventory

Constraint type	Where used	Purpose
PRIMARY KEY	All tables; Book_Authors uses (book_id, author_id)	Guarantees each row has a unique identifier and disallows NULL key values.
FOREIGN KEY	Books.category_id; Book_Authors.book_id/author_id; Loans.book_id/member_id	Prevents child rows from referencing missing parent records.
UNIQUE	Categories.category_name; Books.isbn; Members.email	Prevents duplicate category labels, ISBNs, and member email addresses.
NOT NULL	Required names, IDs, status/date fields, book and loan references	Ensures mandatory attributes are provided.
CHECK	ISBN length; publication year; copy counts; member/loan status; dates	Rejects values that violate explicit business rules.
DEFAULT	Books total_copies/available_copies; Members.status; Loans.status	Supplies a reasonable value when an insert omits the optional defaulted column.
Referential actions	CASCADE for Book_Authors; RESTRICT for loan history references	Cleans relationship bridge rows but protects historical circulation records.

2. Verification queries (expected clean-state results)

```
-- Missing category references from books: expected 0
SELECT COUNT(*) AS orphan_book_categories
FROM Books AS b
LEFT JOIN Categories AS c ON c.category_id = b.category_id
WHERE c.category_id IS NULL;

-- Missing books or members referenced by loans: expected 0
SELECT COUNT(*) AS orphan_loans
FROM Loans AS l
LEFT JOIN Books AS b ON b.book_id = l.book_id
LEFT JOIN Members AS m ON m.member_id = l.member_id
WHERE b.book_id IS NULL OR m.member_id IS NULL;

-- Invalid inventory counts: expected 0
```

```
SELECT COUNT(*) AS copy_count_violations
FROM Books
WHERE total_copies < 1 OR available_copies < 0
   OR available_copies > total_copies;

-- Inconsistent returned status/date combinations: expected 0
SELECT COUNT(*) AS loan_status_violations
FROM Loans
WHERE (status = 'Returned' AND return_date IS NULL)
   OR (status IN ('Issued', 'Overdue') AND return_date IS NOT NULL);

-- Search for duplicate candidate values: all queries should return no rows
SELECT isbn, COUNT(*) AS n FROM Books GROUP BY isbn HAVING COUNT(*) > 1;
SELECT email, COUNT(*) AS n FROM Members GROUP BY email HAVING COUNT(*) > 1;
```

Verification query	Expected result
orphan_book_categories	0
orphan_loans	0
copy_count_violations	0
loan_status_violations	0
duplicate ISBN/email checks	No rows returned

3. Negative tests - each statement should be rejected

Run these statements one at a time after seeding the standard sample data. They are intentionally invalid; do not add them to the main setup script. Exact error wording can differ by MySQL version, but the constraint should reject each row.

```
-- UNIQUE: duplicate ISBN should fail (duplicate entry)
INSERT INTO Books (title, isbn, category_id, publication_year, total_copies, available_copies)
VALUES ('Duplicate ISBN test', '9780132350884', 1, 2020, 1, 1);

-- FOREIGN KEY: category 999 does not exist
INSERT INTO Books (title, isbn, category_id, publication_year, total_copies, available_copies)
VALUES ('Invalid category test', '9781111111111', 999, 2020, 1, 1);

-- CHECK: available copies cannot exceed total copies
INSERT INTO Books (title, isbn, category_id, publication_year, total_copies, available_copies)
VALUES ('Invalid stock test', '9782222222222', 1, 2020, 2, 5);
```

```
-- CHECK: unknown member status should fail
INSERT INTO Members (first_name, last_name, email, phone, membership_date, status)
VALUES ('Invalid', 'Status', 'invalid.status@example.edu', '+91-9000000099', '2026-10-23', 'Gold');

-- FOREIGN KEY: member_id 9999 and book_id 9999 do not exist
INSERT INTO Loans (book_id, member_id, loan_date, due_date, status)
VALUES (9999, 9999, '2026-10-23', '2026-11-06', 'Issued');
```

Expected outcomes: duplicate ISBN is rejected by UNIQUE; nonexistent referenced IDs are rejected by FOREIGN KEY; invalid copy/status values are rejected by CHECK. If a constraint violation is accepted, verify that MySQL version supports and enforces CHECK constraints and confirm that the table uses InnoDB.

4. Test the DEFAULT status safely

```
-- The omitted status should be set to Active; ROLLBACK removes the test row.
START TRANSACTION;
INSERT INTO Members (first_name, last_name, email, phone, membership_date)
VALUES ('Default', 'Check', 'default.check@example.edu', '+91-9000000098', '2026-10-23');
SELECT member_id, email, status
FROM Members WHERE email = 'default.check@example.edu';
ROLLBACK;
```

The SELECT inside the transaction should show status = Active. ROLLBACK ensures the sample member is not left in the database. Auto-increment values can still have gaps after rollback; that is normal.